SYMPTOM SOLVER – Predict your disease

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

To

Guru Gobind Singh Indraprastha University, Delhi

Guide: Ms. Sonia Batra
(Assistant Professor, BCIIT)

Submitted by: Mohd Anas Siddique
Roll No.: 01311104424

Banarsidas Chandiwala Institute of Information Technology
Chandiwala Estate, Maa Anandmai Marg, Kalkaji, New Delhi-110019

Minor Project - II (MCA - 170)
BATCH (2024 - 2026)

Acknowledgment

I would like to express my sincere gratitude to all those who contributed to the completion of this project.

Special appreciation goes to my project supervisor, **Ms. Sonia Batra**, for their guidance and mentorship throughout the project. Their insights and encouragement significantly enriched the quality of our work.

I'd also like to extend my thanks to my Director, **Prof (Dr.) Ravish Saggar,** for granting me this wonderful opportunity to be part of this project.

My gratitude extends to all our staff members for their invaluable assistance, guidance, and encouragement, which made this project possible.

My appreciation extends to the library staff for their vital support in accessing research materials.

I'd like to express my heartfelt gratitude to **Guru Gobind Singh Indraprastha University** for affording us the opportunity to collaborate on the **Symptom Solver – Predict your disease**.

Lastly, I want to acknowledge my family and friends for their unwavering support during the course of this project. Their encouragement kept me motivated, and their understanding during busy times was truly appreciated.

Thank you to everyone who played a part in making this project a success.

Mohd Anas Siddique

01311104424

Certificate

This is to certify that this project entitled **"SYMPTOM SOLVER – Predict your disease"** submitted in partial fulfillment of the degree of Master of Computer Applications (MCA) to the **"Guru Gobind Singh Indraprastha University"**, done by **Mr. Mohd Anas Siddique**, **01311104424**, is an authentic work carried out by him under my guidance.

The matter embodied in this project work has not been submitted earlier for award of any degree to the best of my knowledge and belief.

_____ _____

Signature of the student **Signature of the Guide**

Table of Contents

SYNOPSIS: Symptom Solver

1. STATEMENT ABOUT THE PROBLEM:

In contemporary healthcare, the challenge of providing personalized medication recommendations is exacerbated by the vast array of pharmaceuticals available and the intricate nature of individual patient needs. This project addresses the need for an efficient system that can analyze patient data and suggest appropriate medications to enhance treatment outcomes while minimizing adverse effects. Problem Points:

● Difficulty in selecting the right medication for diverse patient needs.

● Increased risks of incorrect prescriptions leading to inefficiencies and adverse outcomes.

● A time-consuming process for healthcare professionals to assess treatment options.

2. OBJECTIVE AND SCOPE:

The primary objective of this project is to develop a medicine recommendation system that leverages Python programming to process patient information and recommend suitable medications based on established guidelines and patient-specific characteristics.

Scope:

● Target Audience: Healthcare professionals and patients.

Key Features:

● Medication recommendations based on symptoms.

● Integration of AI/ML models to make predictions based on input data.

● Evaluation of model performance to ensure accuracy.

Limitations:

● This is a prototype and should not replace professional medical advice.

● The dataset is limited, and more real-world data may be needed for better results.

3. SIGNIFICANCE OF TOPIC CHOSEN:

The selection of this topic is motivated by the increasing demand for precision medicine and the potential of technology to improve healthcare delivery. By automating the recommendation process, the system aims to reduce the cognitive load on healthcare professionals and mitigate the risk of medication errors, ultimately leading to improved patient safety and care.

4. PROCESS DESCRIPTION:

The development process involves collecting data from multiple sources, including medical databases and clinical guidelines. This data will be processed through algorithms that evaluate patient profiles against medication efficacy and safety parameters. The end result will be an intuitive application that presents healthcare providers with actionable insights. Key Steps in the Process:

1. Data Collection: Using the Kaggle dataset that includes symptoms, diseases, and corresponding medicines.

2. Data Preprocessing: Cleaning the data by handling missing values and ensuring that it's properly formatted for machine learning models.

3. Model Development: Implementing machine learning algorithms such as decision trees, random forests, or KNN to analyze the dataset.

4. Model Evaluation: Evaluating the model using accuracy, precision, recall, and F1-score to ensure its reliability.

5. Deployment: Creating a web interface or API for users to input patient data and receive medication recommendations.

5. METHODOLOGY:

This project will employ a systematic methodology that includes data collection, algorithm development, system design, and user testing. Python libraries such as Pandas for data manipulation, Scikit-learn for machine learning, and JavaScript for web application development will be utilized to facilitate the project's execution.

1. Data Preprocessing: ○ Clean the data by removing duplicates, handling missing values, and normalizing features.

2. Feature Engineering: ○ Select relevant features like symptoms and medical history, which will help in making accurate predictions.

3. Model Training: ○ Use algorithms like decision trees or random forests to train the system.

4. Evaluation: ○ Evaluate the model using standard metrics to measure its performance in making accurate recommendations.

5. Implementation: ○ Implement the recommendation system in a simple user interface, where healthcare professionals can enter patient data to get suggestions.

6. HARDWARE AND SOFTWARE TO BE USED:

The project will be developed using a standard computer system equipped with Python and relevant libraries. Software tools such as Jupyter Notebook for coding and testing, along with a

database management system for storing patient data, will also be employed. The system will be hosted on a local server. Hardware Requirements:

● A computer or laptop with a minimum of 4GB of RAM.

● Stable internet connection for accessing the dataset and deploying the system.

Software Requirements:

● Programming Languages: Python

● Libraries and Tools: ○ Pandas, NumPy (for data handling)

○ Scikit-learn (for implementing machine learning models)

○ HTML and JS

● Development Environment:

○ Jupyter Notebook, VS Code, or PyCharm for coding and model development.

○ Kaggle for dataset access.

7. KEY CONTRIBUTIONS OF THE STUDY:

This project aims to make significant contributions to the field of medical informatics by providing a scalable solution for medicine recommendation that can be adapted for various healthcare settings. It seeks to enhance the decision-making process for healthcare providers, thereby improving patient treatment plans and health outcomes.

8. CONCLUSION:

In conclusion, the Medicine Recommendation System using Python represents a pivotal advancement in the intersection of technology and healthcare. By harnessing the power of data

analytics, the project aspires to streamline the medication prescribing process, reduce errors, and ultimately foster a more effective healthcare environment. This initiative stands to benefit both practitioners and patients, paving the way for future innovations in personalized medicine.

Main Report

I. Objective & Scope of the Project:

Objective:

To create a program which can be used to analyse a user's symptoms and predict the disease based on the symptoms.

1. To create an intelligent assistant capable of predicting diseases from a list of symptoms.
2. To suggest preventive measures, medication, diets, and workouts.
3. Aimed at early diagnosis and awareness of common diseases.
4. Applicable for health organizations, individual users, or telehealth platforms.

Scope:

The scope of this project includes:

1. User Form for inputting the symptoms and get the result.
2. Python Libraries to build the model and predict the disease.
3. The application will be deployed and accessible via a web browser.

The project does not include features such as:

- Backend or database connection for storing user data.
- Advanced analytics or machine learning-driven suggestions.

II. Theoretical Background & Definition of the Problem:

Introduction:

Having a doctor or pharmacist consultation is a must when getting sick. Generally, a doctor is one of the professions in the healthcare field, who is responsible for both general and specialty disease diagnosis.

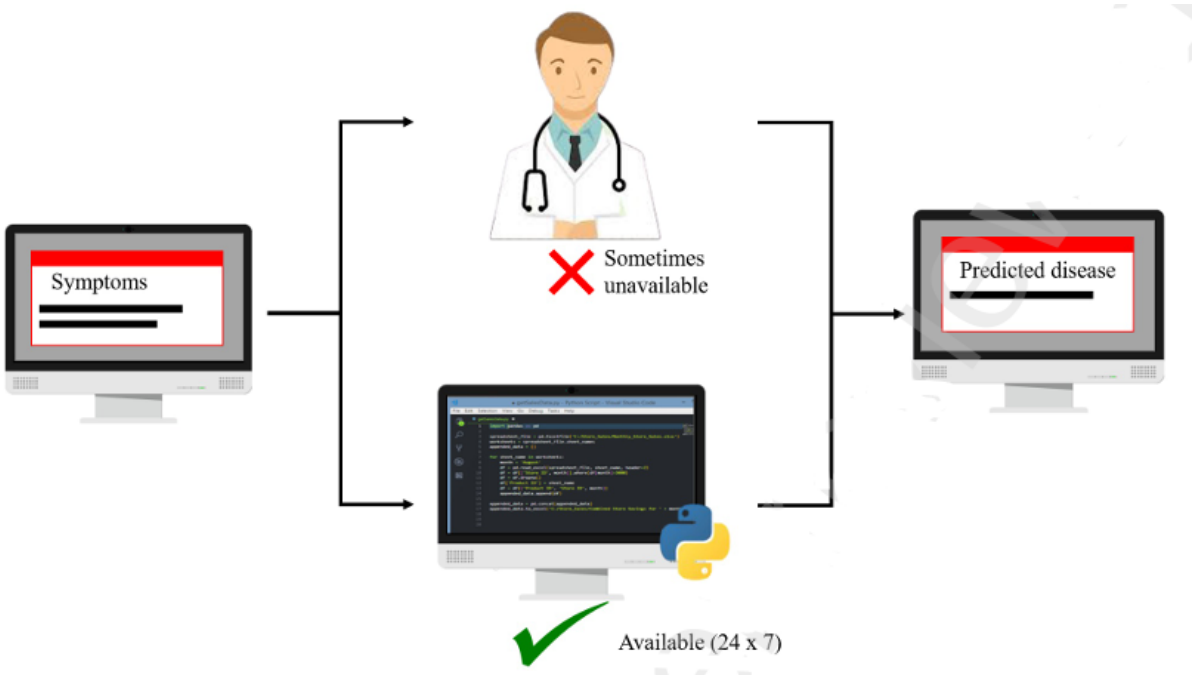


Figure 1: Symptom-Based Disease Prediction Using Doctor Consultation and AI Assistance

However, there is a problem where patients may not access healthcare services when they met some common clinical diseases such as dengue, chickenpox, allergy, and so forth. Transportation issues, the amount spent on medical services, as well as the low health awareness of patients themselves may be the major issues that cause the problems mentioned to have happened. Other than pharmacists and doctors, people are facing a challenge where he or she does not have any experiences when coming to the diagnosis of clinical diseases such as dengue, chickenpox, allergy, and so forth. In the 21st century era of technology, machine learning has

evolved and is widely used in various industries, including agriculture, finance, travel, and so forth. One of the significant fields which involves machine learning is the medical field.

Different kinds of machine learning approaches have been used in the medical industry, such as natural language processing to create a medical diagnosis chatbot, image processing to determine rare disease, an expert system to determine general disease, and so on. Since the project relates to prediction in the healthcare field, therefore machine learning will be used for prediction in the project. Based on the problems stated in the first paragraph, an offline-based medical diagnosis system is proposed to help patients in identifying diseases themselves. After selecting symptoms, they will receive an accurate diagnosis result from the system.

In short, patients can know the disease they have and later decide on whether there is a need to go for doctor consultation. In order to produce an accurate diagnosis result, a set of data will be trained by using appropriate machine learning algorithms. After testing part of the data for its accuracy, data (symptoms) entered by the patient will be input and the outcome will be produced based on the data trained using machine learning algorithms. The dataset collected would be the medical diagnosis records of other patients based on their symptoms.

Definition of the Problem:

Consulting a doctor may cause someone to have huge spending on it. According to the research found, the healthcare cost of the United States (US) has risen from $2879 billion to $3492.10 billion in five-year time (from the year of 2013 to 2017).

Also, according to Deloitte (2019), the spending of worldwide medical care is presumed to be increased continuously from the year of 2017 to 2022, with an annual rate of 5.4%. The rising cost in the medical field causes part of the people not able to pay for the clinical or hospital consultation, especially for the lower-income group, and this causes more people choose to not

seek for medical care when they are having clinical diseases such as allergy, high fever, and so forth. When diseases getting worsen, the chance of getting effective treatment may be lower. There is a problem where people nowadays have low health awareness on themselves. As a worker, he or she may not willing to consult a doctor when getting sick, as there are tons of jobs needed to be done by them every day. Therefore, health awareness of most of the people become lower, which may cause the disease to become more serious in the future. Besides, as mentioned in the first paragraph, consulting a doctor may need to spend a lot on it. This may lead to the same problem where health awareness becomes lower and causing the disease to get more serious. Going to a physical clinic or hospital for doctor consultation may be hard for people who live in rural areas. According to research, the number of medical clinics or hospitals in rural areas is much lesser compared to in urban areas. This will lead to a problem where people who live in rural areas have fewer chances in obtaining medical care they need. If rural residents want to access a particular medical service, they may need to have their transportation to reach the medical service, in which the medical service may locate far from home.

AIM:

To come out with an implementation of a symptom-based disease prediction system using machine learning algorithm to bring convenience to users by identifying the clinical disease themselves based on a set of past data

OBJECTIVES:

The project objectives include: -

- To have an investigation of few similar existing systems.
- To identify the problems of each similar existing system so that these problems can be improved within the proposed system.

- To compare various machine learning algorithms and identify the most suitable algorithm for prediction.
- To evaluate each chosen algorithm in terms of its accuracy and choose the best one to be implemented within the system.

Background:

According to the research, healthcare service acts as the biggest and one of the most important industries in the United States. Various treatments have been offered to patients through the access to medical services, and patients are able to get consultations from healthcare professions such as pharmacist and doctor. According to one of the reports found, the worldwide healthcare market is growing continuously by reaching a value of approximately $8452 billion in the year of 2018, and it is expected to grow to approximately $11908.9 billion in the year of 2022. From here, a conclusion can be made that healthcare services may be one of the industries that can be further explored and improved with the existence of technologies in this 21st century, which is the era of science and technology. However, there is still part of the people who choose to not accept treatment from the healthcare profession and their health condition is deteriorating at the end. Therefore, it is assumed that these people may have a few concerns before getting treatment. A study was carried out to discover the reasons why patients chose to avoid from accessing to medical service. From the study, it can be discovered that there are actually multiple aspects which make the respondents to avoid from getting themselves a medical care, including interpersonal factor such as communication issues, traditional barrier such as transportation difficulties, as well as organizational factor such as long waiting time. To solve these problems, an in-depth research is carried out by exploring whether or not the evolvement of technology in the medical field can help in solving the problems stated. As mentioned above, technology has been quietly and slowly evolved in the healthcare industry in this era of new

technology. One of the reports mentioned that the market growth of smart healthcare industry is expected to reach a CAGR of 24.1% between the year of 2019 and 2023. From here, it can be summarized that the medical industry will continuously adopt smart technology now and in the future. Smart healthcare industry can be said as the combination of either big data, internet of things (IoT) or machine learning, as well as deep learning with the healthcare system. Using a smart healthcare system enables the patients to not only have the option of accessing physical hospital, but also the option to access to the healthcare system anywhere at any time without any boundaries. From here, it can be known that both issues (transportation difficulties and long waiting times) can be solved through the smart healthcare system, as it allows patients to access to healthcare service anytime at anywhere. Besides, the smart healthcare system allows the cost of medical to be reduced as well. All in all, it can be concluded that the utilization of technology in healthcare industry is able to solve all the problem contexts mentioned.

Manual Doctor Consultation Process:

Due to privacy issue, secondary research is carried out instead of primary research to understand the manual process of doctor consultation, so that the proposed system is able to mimic the procedure well, but with a simplified process.

A study was carried out by an author to investigate the time taken for the patient to wait for doctor consultation as well as the time taken for the patient to consult a doctor at a clinic so that the author can produce a few strategies to improve this matter.

Another research was carried out by an author to improve the old system of doctor consultation by designing an electronic consultation system. From both of these journals, the researcher is able to know and understand the walk-in process has to go through a lot of processes, including registration, pre-consultation, consultation, booking of appointment, payment process, as well

as pharmacy process (wait for the medicine to be given by the doctor). From here, it is assumed that this is one of the reasons why some of the people think that they have to spend a lot of time on manual doctor consultation, as there are a lot of processes to be gone through.

III. System Analysis & Design vis-a-vis User Requirements

System Analysis:

To understand user requirements, we conducted surveys and interviews with potential users. The key requirements identified were:

User Requirements:

- Patients: Easy symptom input, accurate prediction
- Doctors: Reference tool for diagnosis

Functional requirements:

- Symptom Input form
- Prediction Engine
- Disease information display

Non Functional Requirements:

- Fast response time
- Data handling

From these requirements, the system architecture was designed to fulfill the needs of users with a simple, efficient interface for browsing properties.

System Design:

The system design for the project includes:

1. **Frontend**: Flask being used to create the python app frontend and give the user interface to the users.

2. **Python**: Python Libraries used to split dataset into train, test partition and build the model for predicting disease.

Key features in the design:

- A homepage that displays a form for symptoms.
- A result section which shows the predicted disease.
- A bottom section displaying the diets, precautions and medications, etc.

Introduction to Machine Learning:

There are a lot of subsets involved in artificial intelligence, and one of them would be machine learning. Machine learning has been widely used in various industries, including medical, agriculture, manufacturing, sales, and so on.

By using machine learning, it provides the opportunity for the system to learn from a chunk of past data itself and improve by the experiences automatically. One of the functions of machine learning is that it acts as an assistant within data analytic field. There are various form of data analytics, including descriptive analytic, predictive analytic, as well as prescriptive analytics. In predictive analytics, machine learning algorithm is used to analyze the past data collected and to predict future output. Generally, machine learning allows the system to learn by looking at the data such as the pattern of numeric data, instructions, and so forth. After the data is being learned or trained, it allows the machine to make a better decision in the future.

There are three types of machine learning, including unsupervised learning, supervised learning, as well as reinforcement learning. Supervised learning learns through a set of existing data consisting of both input data and output data, in which the output data is the correct answer. The learning process will be terminated when the algorithm reaches a good

performance. There are two common groups that fall under supervised learning, including classification and regression. Classification problem produces an output with a category, such as “yes” or “no”; while regression problem produces an output with an exact value, such as “height”. On the other hand, unsupervised learning refers to a machine learning algorithm that aims to search for unknown patterns or structures in the data, from a set of data that has input data but without any labeled output. Using unsupervised learning allows the data to be categorized based on their similarities. There are two common groups that fall under unsupervised learning, including clustering and association. Clustering aims to find instinctive groups within a set of data, such as customer grouping according to their purchasing behavior; while association aims to explore the rules which outline big chunks of data.

Machine Learning in Medical Field:

A lot of applications have been developed with the technology of machine learning. One of its significant applications would be within the medical field. Machine learning is getting more and more popular in today’s world, as it is the latest trend in enhancing the healthcare system. Somehow in achieving the goal of reducing the cost of healthcare, machine learning may prove to be one of the solutions. According to the research, the healthcare industry uses machine learning to diagnose a particular disease, either using patients’ past records as data to predict the future outcome by entering few variables’ values, or by using image processing to scan, process, and determine the future outcome.

Machine learning allows physicians to carry out a nearly perfect diagnosis if it is applied and utilized effectively. One of the main advantages of using machine learning to predict the outcome of a particular disease is that recommendation can be made according to someone’s health conditions such as his or her age, gender, and so forth, and this may assist someone to

have much more health awareness on him or herself at the first stage. Thus, this advantage is one of the reasons encountered in solving the problem of this proposed project.

Comparison of models:

As the proposed system will classify the output into multiple classes based on selected symptoms, therefore classification algorithm will be used. Multiple types of algorithms that can be used in solving classification problems, including support vector machines, Naïve Bayes classifier, decision trees, random forest, and so forth. From the research found, a conclusion can be drawn that every developer has applied different types of supervised learning algorithms during the implementation of the prediction system. This can be explained by the journal which aimed to compare different types of classification algorithms to detect network intrusion. From the research, it can be seen that the combination of random forest and support vector machine produce the highest accuracy compared to other algorithms, such as the combination of support vector machine and BayesNet, support vector machine and logistic regression, and many more. Another project had been done to evaluate the main reason which affects the performance of newly listed companies using support vector machine as well as various decision tree models. Results showed that one of the decision tree models named C5.0 had achieved the highest accuracy with 96.46% compared to other models.

Therefore, the researcher decided to choose three common types of multiclass classification algorithms for further investigation, including Naïve Bayes and Support Vector Machine. Also, these two algorithms will be used to test the accuracy during the implementation, and the one with the highest accuracy will be selected to be used in the prediction process.

Table 1: Comparison of Naive Bayes and Support Vector Machine

	Naïve Bayes	**Support Vector Machine**
Advantage (s)	Simple prediction. Well performed in multiclass classification. Less computational time is needed for the training process.	Works efficiently if the separation between multiple classes are clear. Memory efficient. Works well in high dimensional spaces.
Disadvantage (s)	Require large dataset to get better prediction result	Time consuming. Needs huge matrix operations.

IV. System Planning (PERT Chart):

Here is the PERT chart that outlines the planning process for the Host Haven project:

1. **Requirement Gathering** (Start)
2. **System Design** (Load datasets, clean and process data)
3. **Frontend Development** (Create form and other interface elements using Flask)
4. **Python** (Building models for prediction)
5. **Integration & Testing** (Integrating functions and testing the feature to predict)
6. **Deployment** (Deploy to a cloud platform)

Table 2: System Planning with task description and estimated time

Event	Task Description	Dependencies	Time Estimate
A	Data Loading, Cleaning and preprocessing	None	2 days
B	Train, Test Split and labeling the target feature	A	3 days
C	Building and Comparing machine learning models	B	1.5 weeks
D	Picking a model and create function for disease prediction using user's symptoms	C	2 weeks
E	Cleaning other datasets and map predicted disease to find diets, medications, and precautions	B, D	1 week
F	Building Flask application using pycharm	E, D	1.5 week
G	Integration and Testing	F	1 week

V. Methodology Adopted, System Implementation & Details of Hardware & Software Used, System Maintenance & Evaluation

Methodology Adopted:

System development methodology refers to a framework that can be used in system development planning, managing, and controlling. There are various types of methodologies that evolved with their strengths and weaknesses, and the selection of suitable methodology needs to look into multiple aspects of a project, such as project size, project development duration, project team size, and so forth. Two different methodologies will be compared, and the best one will be used to carry out the project. According to the research, agile digital transformation, extreme programming, and rapid application development are the three top methodologies used in software development process in the year of 2018 and they are estimated to be the best methodologies in software development process in the year of 2019. In conjunction with the project scenario, both extreme programming and rapid application development are selected to be further compared. After comparing these methodologies, rapid application development (RAD) is selected to be implemented in the proposed project.

Machine Learning Model:

Step 1: problem identification:

As mentioned in the literature review section, there are few steps in implementing machine learning model within the system. First, to define problem of the project. The problem has been identified earlier, which aim to help the patient in improving their convenience by making disease prediction themselves anytime at anywhere.

Step 2: identification of required data:

In step 2, the required data which will be helpful in achieving the project objective is identified. Since the project aimed to help patients in predicting possible disease themselves, thus the dataset used would be the most suitable one to carry out machine learning model implementation. The dataset is named as disease prediction dataset, consisting of 131 symptoms and 41 types of diseases, and it is retrieved by the author from one of the hospitals. The dataset is a multivariate dataset consisting of structured data. It is an open-source dataset that can be used by anyone. The source of the dataset is from the Kaggle website, which is one of the most reliable sources for the data scientist to get tools and resources to achieve the research goal.

Step 3: feature selection:

In this step, the important features will be selected in implementing machine learning model, meaning the variable which is helpful in solving the project's problem will only be selected, while the useless variable will be eliminated. Figure 7 above shows part of the independent variables in the dataset. Each independent represents one symptom, therefore all attributes in the dataset is important, as it will be used in predicting the disease. In short, all the attributes in the dataset will be used as the outcome will need to depend on these independent variables heavily.

Step 4: data pre-processing:

In this step, the data will be pre-processed into a proper format so that it can be understood by the machine. These steps include cleaning noisy data, missing data, as well as transforming the data into a proper data format if there are any attributes is in an improper format.

By having the summary of one of the attributes named “itching”. It is clear that it only contains binary value, which is 0 and 1, same goes to the other attributes. Since there is no noisy data in the dataset, thus there is no need to do any further processing to clean the noisy data. Also, it can be seen that there is no missing value for each and every “cell” in the dataset. Thus, there is no need to do any processing to clean missing data.

Step 5: data splitting:

After pre-processed data, the next step would be data splitting. In this step, all the data will be split randomly by the machine into training and testing set with the specification of data splitting ratio. The purpose of splitting data is that split data with a larger ratio will be used in training the machine learning model, while data with a lower ratio will be used in testing the machine learning model, to see whether or not the trained model performs well by looking at the accuracy. If it is not performed well, the model will be retrained.

Step 6: implementation of machine learning model:

Machine learning model will be implemented in this stage. There are 3 appropriate types of multiclass classification algorithms being identified in the literature review section, including Naïve Bayes and Support Vector Machine. Therefore, these 2 machine learning algorithms will be used for training along with the pre-processed dataset together. For Naïve Bayes algorithm, there are 3 various types of algorithms, and each algorithm targets different objectives. After researching, Multinomial Naïve Bayes is used, as this type of Naïve Bayes

algorithm deals with the binary input. Since all the input of dataset chosen is in binary form, therefore Multinomial Naïve Bayes is selected.

Step 8: save model:

l After evaluating the two models, the model having the highest accuracy among the two is saved. Thus, the model will be saved into a file called pickle and will be used during the disease prediction process.

System Implementation:

Frontend: Flask

Backend: Python (Scikit-learn, Pandas)

Database: CSV

Hardware & Software:

Hardware: Development was performed on a personal laptop (8GB RAM, i3 Processor).

Software: Python 3.8+, Google Colab, and PyCharm.

VI. Detailed Life Cycle of the Project:

a. ERD (Entity Relationship Diagram):

The ERD visualizes how entities like Disease, Diets, Medications, Precautions and Workouts are structured and related. For example:

- A Disease can have diet items in the table diets.
- Each disease has a description which is unique.

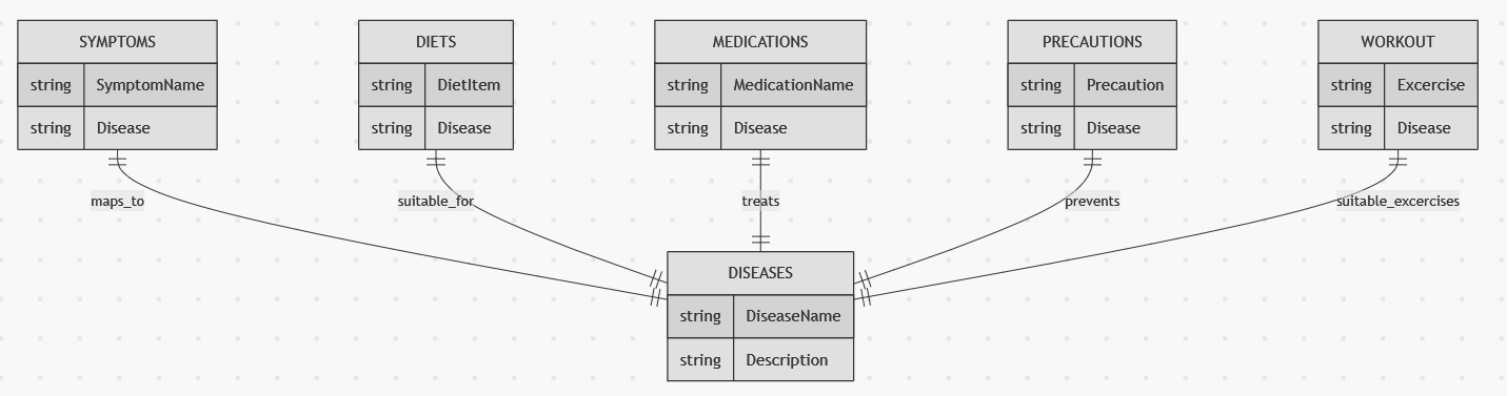


Figure 2: Entity Relationship Diagram of the system

b. DFD (Data Flow Diagram):

Level 0:

The DFD shows how data moves within the system. For example:

- The user inputs the symptoms and symptoms are passed to the system.
- The system then analyzes, and then gives the result.

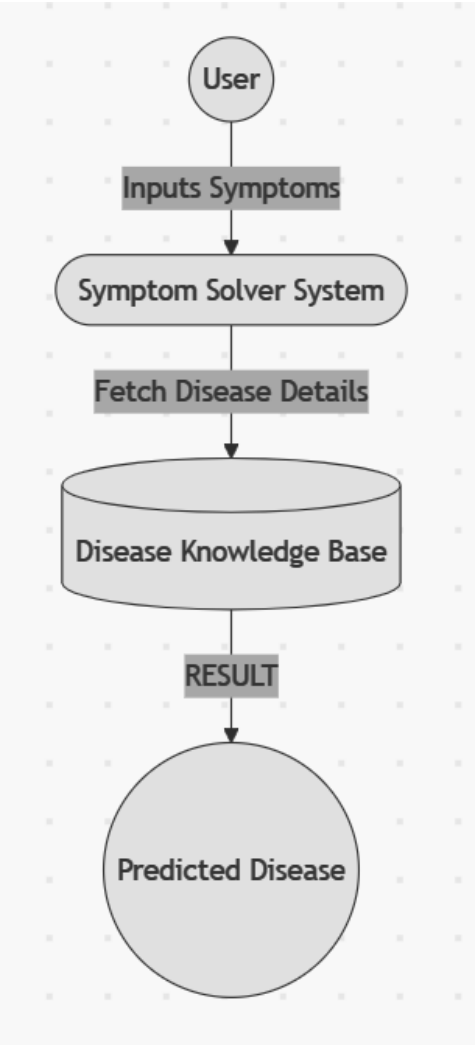


Figure 3: Level 0 Data Flow Diagram of the system

Level 1:

- The user inputs the form and the symptoms are passed to the system.
- The system checks if the symptoms are valid or not.
- If valid, it uses the model and predicts the disease and displays other results such as medications, diets and precautions, etc.
- If symptoms are not valid, it then shows an error message and the user tries again.

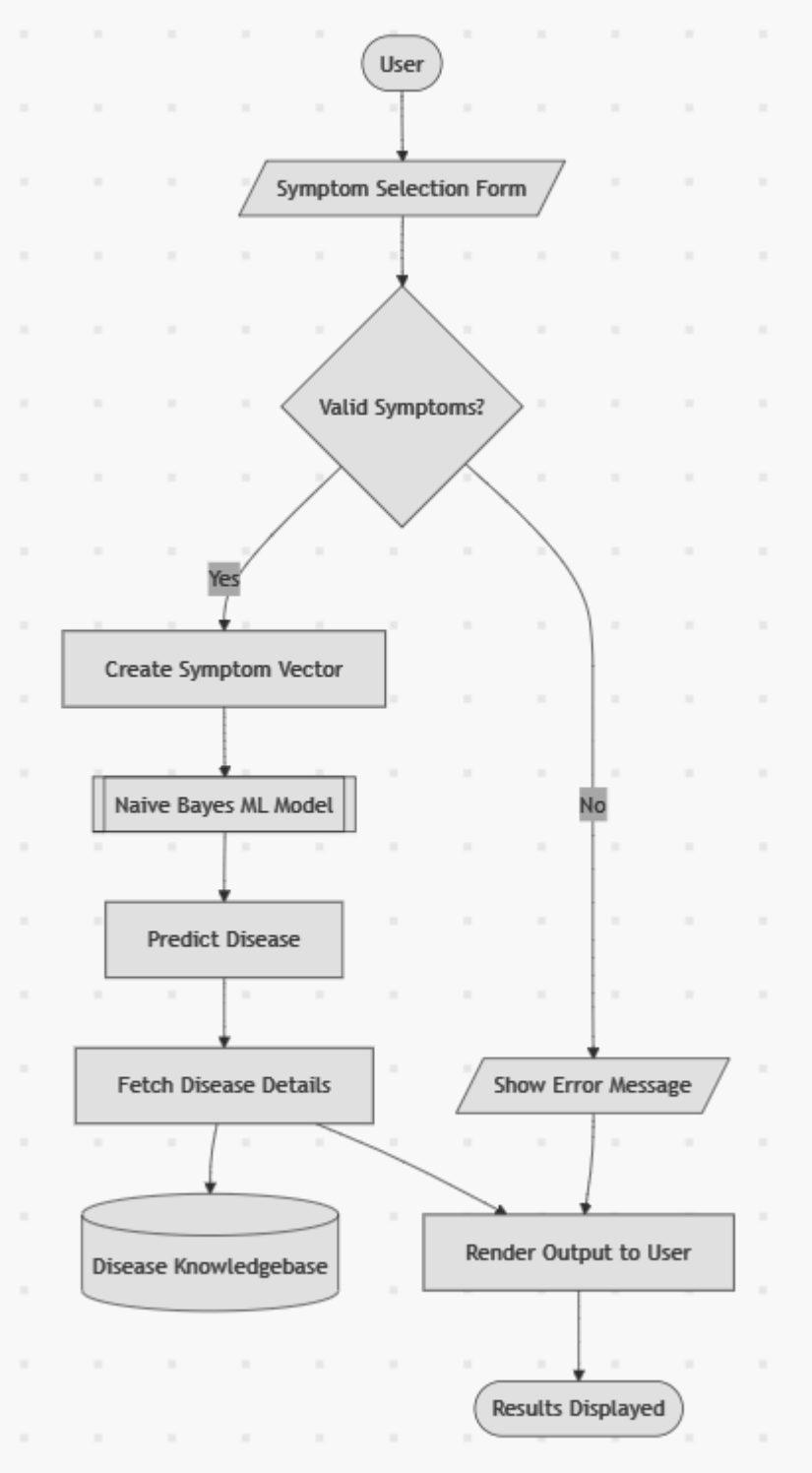


Figure 4: Level 1 Data Flow Diagram of the system

c. Flow Chart:

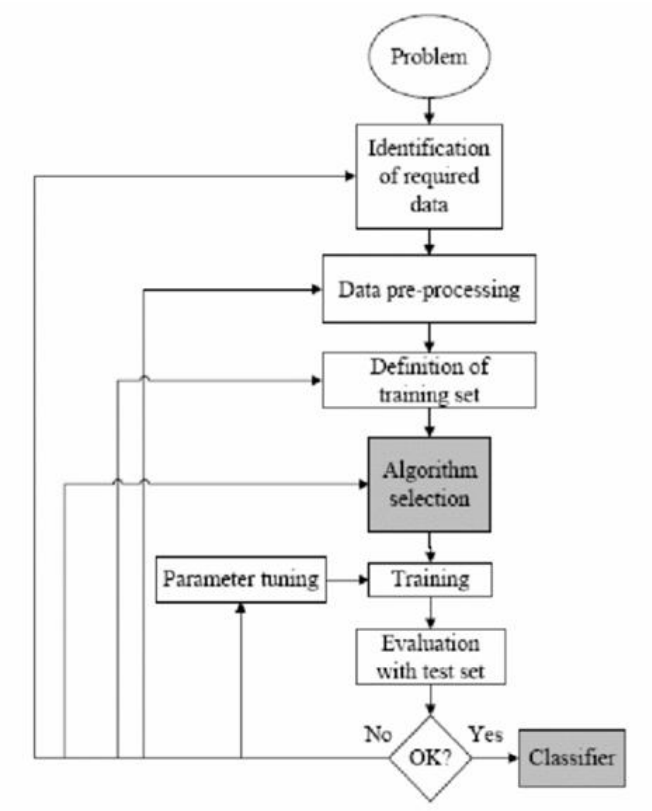


Figure 5: Flow Chart Diagram of the system

The first phase is the problem identification, which means that a specific problem has to be identified before training the algorithm. For instance, figuring out whether an email is a spam email. After identifying the problem, related data has to be identified and gathered, as the data will be used to train the supervised learning algorithm

Data collection is said to be one of the most significant procedures as the data's quantity as well as quality will affect the performances of the model. The third phase would be the pre-processing of the data gathered. In this phase, the collected data will be loaded into an appropriate place and it will then be prepared to be used for the training process. From this phase, it is time for the developer to see if there are any relationships between various variables. For instance, the developer can check whether the number of output A is more than

output B. If so, the result generated would seem to be biased as the model will know that output A is correct at most of the time. Therefore, the developer has to organize the data well so that the biases of the result will not happen. The gathered data will be separated into two sections, one will be used in the training process, while another one will be used in the testing process to judge the performance of the trained model. The reason why the separation of data is carried out is to ensure the model would not only memorize the questions, but it is able to give an accurate result given different input.

Model selection will be carried out after pre processing the data and defining both training and testing data. Model selection is the most important process among all, as different types of models will be used for different purposes. For instance, some models are suitable for image processing, while some models are suitable for data in numerical form. After selecting the most appropriate model, data training process will be carried out by using the selected model. The model will keep improving by keep training the model until it has the ability to get a high accuracy result. In the training process, some random values will be given for weights and biases, and these values will be used to predict the output. If the predicted output is not achieving high accuracy, the values of weights and biases will be kept adjusted until a high accuracy of prediction is achieved.

Evaluation is carried out when the training process is done to see whether the trained model is good enough by inputting a set of test data (which is one of the two parts separated out in the previous process). This process allows the developer to see the performance level of the trained model. If the predicted output is far different from the real output, then the developer

will need to re-train the model. In contrast, if the predicted output is accurate enough, then the model is ready to be used for classification.

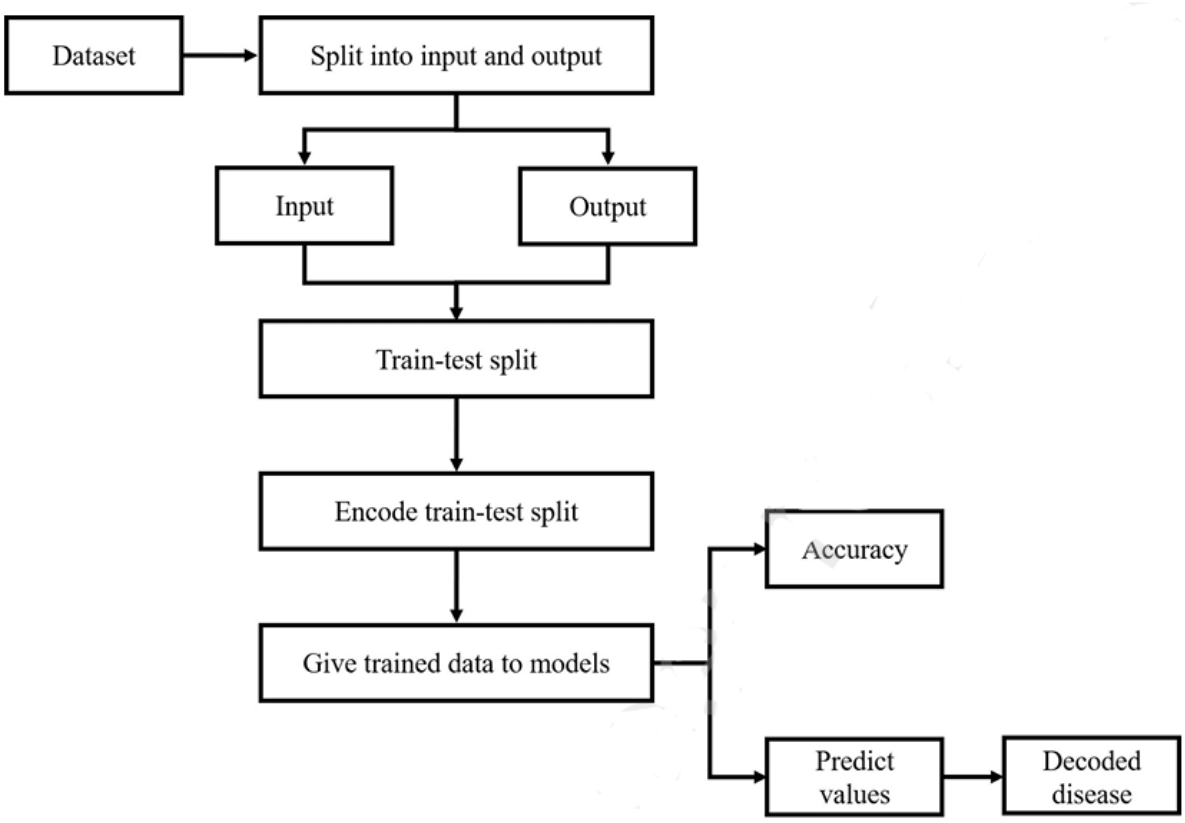


Figure 6: Workflow of Machine Learning-Based Disease Prediction System

d. Input and Output Screen Design:

Input Screen:

- A form which enables the user to input the symptoms to analyse and predict the disease.

Output Screen:

- A section which includes the predicted disease name, description, diets, medications, precautions and workouts for the same.

e. Processes Involved:

1. **Prediction Process**: The user inputs symptoms into form.
2. **Display Process**: The results are shown on the frontend.

f. Methodology Used for Testing:

Testing was done using both manual and automated approaches:

- **Manual Testing**: The application was tested for usability and functionality across different devices and browsers.
- **Automated Testing**: Unit tests were written using Jest for backend API endpoints.

g. Testing Report:

Test Objectives:

- Validate that the user can enter the symptoms correctly without any issue.
- Ensure symptoms are being passed to the function and the model.
- Verify that if the symptom is not valid, then it shows an error message.
- Verify if the symptom is valid, then it predicts the disease and displays the other information also.
- Check the results are getting displayed on the screen.

Test Scenarios and Cases

1. **Verify user can input symptoms**
    - **Scenario 1**: Successful input symptoms
        - **Test Case 1.1**: Enter a symptom.
        - **Test Case 1.2**: Submit with missing field.
2. **Check model receives the symptoms properly**
    - **Scenario 2**: Passing symptoms to model

- **Test Case 2.1**: Model gets the symptom from the field.
- **Test Case 2.2**: Empty symptom field from the user..

3. **Verify symptom is not valid**
    - **Scenario 3**: Verifying symptoms.
        - **Test Case 3.1**: If symptom is entered.
        - **Test Case 3.2**: If field has no value.
        - **Test Case 3.3**: If the field has some value, other than symptoms.
4. **Verify symptom is valid**
    - **Scenario 4**: Check valid symptoms.
        - **Test Case 4.1**: Input is a valid symptom.
        - **Test Case 4.1**: Input is empty.
5. **Integration Testing**
    - **Scenario 5**: Integrating whole modules for testing.
        - **Test Case 5.1**: Testing input with symptoms and getting results displayed.
        - **Test Case 5.1**: Testing input with empty field and getting error message to try again.

Test Results:

Table 3: Testing scenarios and test cases

Scenario	Test Case	Status	Comments
Scenario 1	Test Case 1.1	Passed	Input field accepts the symptoms
	Test Case 1.2	Passed	Missing field also.
Scenario 2	Test Case 2.1	Passed	Model gets the input

	Test Case 2.2	Passed	Model gets the empty field also.
Scenario 3	Test Case 3.1	Passed	Symptom is not valid
	Test Case 3.2	Passed	Missing field handled
	Test Case 3.3	Passed	Symptom is not valid
Scenario 4	Test Case 4.1	Passed	Valid symptom
	Test Case 4.2	Passed	Not valid, empty
Scenario 5	Test Case 5.1	Passed	Getting predicted results
	Test Case 5.2	Passed	Displaying error message.

Coding

File: projectSystem.ipynb

```
##Loading Dataset

import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings("ignore")

df=pd.read_csv("Training.csv")

df.head()
"""Number of unique diseases in target column: prognosis"""

len(df['prognosis'].unique())

"""Names of unique diseases in target column: prognosis"""

df['prognosis'].unique()

"""##Train, Test and Split"""

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

X = df.drop("prognosis", axis=1)

y = df['prognosis']

from sklearn.feature_selection import SelectKBest, chi2
from sklearn.model_selection import train_test_split

# Separate features and target
X = df.drop('prognosis', axis=1)
df['prognosis'] = df['prognosis'].str.strip()
y = df['prognosis']

# Select top 15 features using chi-squared test
selector = SelectKBest(score_func=chi2, k=15)
X_new = selector.fit_transform(X, y)

# Get the names of selected features
selected_features = X.columns[selector.get_support()].tolist()

# Create a new DataFrame with only selected features
X_selected = X[selected_features]

# Split the selected data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y, test_size=0.2, random_state=42
)
selected_features

df['prognosis'].unique()

le=LabelEncoder()
```

```
le.fit(y)
Y=le.transform(y)

label_mapping = {index: label for index, label in enumerate(le.classes_)}
label_mapping

X_train,X_test,y_train,y_test  = train_test_split(X,Y, test_size=0.3,
random_state=20)

X_train.shape,X_test.shape,y_train.shape,y_test.shape


"""##Training Using SVM Support Vector Machine"""

from sklearn.svm import SVC
svc=SVC(kernel='linear')
svc.fit(X_train,y_train)
ypred=svc.predict(X_test)
# accuracy_score(y_test,ypred)

from sklearn.naive_bayes import MultinomialNB
mnb=MultinomialNB()
mnb.fit(X_train, y_train)
ypred=mnb.predict(X_test)

#Save model
import pickle
pickle.dump(mnb,open("mnb.pkl",'wb'))

"""##Single Testing

Test 1
"""

print("Predicted Label: ",svc.predict(X_test.iloc[0].values.reshape(1,-1)))
print("Actual Label: ",y_test[0])

"""Test 2"""

print("Predicted Label: ",svc.predict(X_test.iloc[1].values.reshape(1,-1)))
print("Actual Label: ",y_test[1])

"""##Loading other datasets"""

description=pd.read_csv("description.csv")
precaution=pd.read_csv("precautions_df.csv")
workout=pd.read_csv("workout_df.csv")
medication=pd.read_csv("medications.csv")
diets=pd.read_csv("diets.csv")

description.head()

precaution.head()

workout.head()

medication.head()

workout.head()
```

```
"""##Function for symptoms"""

def get_info(dis):
  desc=description[description['Disease']==dis]['Description']
  desc=" ".join([x for x in desc])


pr=precaution[precaution['Disease']==dis][['Precaution_1','Precaution_2','P
recaution_3','Precaution_4']]
  pr=[x for x in pr.values]

  medic=medication[medication['Disease']==dis]['Medication']
  medic=[x for x in medic.values]

  diet=diets[diets['Disease']==dis]['Diet']
  diet=[x for x in diet.values]

  work=workout[workout['disease']==dis]['workout']

  return desc,pr,medic,diet,work
```

```
symptoms_dict = {'itching': 0, 'skin_rash': 1, 'nodal_skin_eruptions': 2,
'continuous_sneezing': 3, 'shivering': 4, 'chills': 5, 'joint_pain': 6,
'stomach_pain': 7, 'acidity': 8, 'ulcers_on_tongue': 9, 'muscle_wasting':
10, 'vomiting': 11, 'burning_micturition': 12, 'spotting_ urination': 13,
'fatigue': 14, 'weight_gain': 15, 'anxiety': 16, 'cold_hands_and_feets':
17, 'mood_swings': 18, 'weight_loss': 19, 'restlessness': 20, 'lethargy':
21, 'patches_in_throat': 22, 'irregular_sugar_level': 23, 'cough': 24,
'high_fever': 25, 'sunken_eyes': 26, 'breathlessness': 27, 'sweating': 28,
'dehydration': 29, 'indigestion': 30, 'headache': 31, 'yellowish_skin': 32,
'dark_urine': 33, 'nausea': 34, 'loss_of_appetite': 35,
'pain_behind_the_eyes': 36, 'back_pain': 37, 'constipation': 38,
'abdominal_pain': 39, 'diarrhoea': 40, 'mild_fever': 41, 'yellow_urine':
42, 'yellowing_of_eyes': 43, 'acute_liver_failure': 44, 'fluid_overload':
45, 'swelling_of_stomach': 46, 'swelled_lymph_nodes': 47, 'malaise': 48,
'blurred_and_distorted_vision': 49, 'phlegm': 50, 'throat_irritation': 51,
'redness_of_eyes': 52, 'sinus_pressure': 53, 'runny_nose': 54,
'congestion': 55, 'chest_pain': 56, 'weakness_in_limbs': 57,
'fast_heart_rate': 58, 'pain_during_bowel_movements': 59,
'pain_in_anal_region': 60, 'bloody_stool': 61, 'irritation_in_anus': 62,
'neck_pain': 63, 'dizziness': 64, 'cramps': 65, 'bruising': 66, 'obesity':
67, 'swollen_legs': 68, 'swollen_blood_vessels': 69, 'puffy_face_and_eyes':
70, 'enlarged_thyroid': 71, 'brittle_nails': 72, 'swollen_extremeties': 73,
'excessive_hunger': 74, 'extra_marital_contacts': 75,
'drying_and_tingling_lips': 76, 'slurred_speech': 77, 'knee_pain': 78,
'hip_joint_pain': 79, 'muscle_weakness': 80, 'stiff_neck': 81,
'swelling_joints': 82, 'movement_stiffness': 83, 'spinning_movements': 84,
'loss_of_balance': 85, 'unsteadiness': 86, 'weakness_of_one_body_side': 87,
'loss_of_smell': 88, 'bladder_discomfort': 89, 'foul_smell_of urine': 90,
'continuous_feel_of_urine': 91, 'passage_of_gases': 92, 'internal_itching':
93, 'toxic_look_(typhos)': 94, 'depression': 95, 'irritability': 96,
'muscle_pain': 97, 'altered_sensorium': 98, 'red_spots_over_body': 99,
'belly_pain': 100, 'abnormal_menstruation': 101, 'dischromic _patches':
102, 'watering_from_eyes': 103, 'increased_appetite': 104, 'polyuria': 105,
'family_history': 106, 'mucoid_sputum': 107, 'rusty_sputum': 108,
'lack_of_concentration': 109, 'visual_disturbances': 110,
'receiving_blood_transfusion': 111, 'receiving_unsterile_injections': 112,
'coma': 113, 'stomach_bleeding': 114, 'distention_of_abdomen': 115,
'history_of_alcohol_consumption': 116, 'fluid_overload.1': 117,
'blood_in_sputum': 118, 'prominent_veins_on_calf': 119, 'palpitations':
120, 'painful_walking': 121, 'pus_filled_pimples': 122, 'blackheads': 123,
'scurring': 124, 'skin_peeling': 125, 'silver_like_dusting': 126,
'small_dents_in_nails': 127, 'inflammatory_nails': 128, 'blister': 129,
'red_sore_around_nose': 130, 'yellow_crust_ooze': 131}
diseases_list = {15: 'Fungal infection', 4: 'Allergy', 16: 'GERD', 9:
'Chronic cholestasis', 14: 'Drug Reaction', 33: 'Peptic ulcer diseae', 1:
'AIDS', 12: 'Diabetes', 17: 'Gastroenteritis', 6: 'Bronchial Asthma', 23:
'Hypertension ', 30: 'Migraine', 7: 'Cervical spondylosis', 32: 'Paralysis
(brain hemorrhage)', 28: 'Jaundice', 29: 'Malaria', 8: 'Chicken pox', 11:
'Dengue', 37: 'Typhoid', 40: 'hepatitis A', 19: 'Hepatitis B', 20:
'Hepatitis C', 21: 'Hepatitis D', 22: 'Hepatitis E', 3: 'Alcoholic
hepatitis', 36: 'Tuberculosis', 10: 'Common Cold', 34: 'Pneumonia', 13:
'Dimorphic hemmorhoids(piles)', 18: 'Heart attack', 39: 'Varicose veins',
26: 'Hypothyroidism', 24: 'Hyperthyroidism', 25: 'Hypoglycemia', 31:
'Osteoarthristis', 5: 'Arthritis', 0: '(vertigo) Paroymsal  Positional
Vertigo', 2: 'Acne', 38: 'Urinary tract infection', 35: 'Psoriasis', 27:
'Impetigo'}

symDICT= {'pain_behind_the_eyes': 0,
 'throat_irritation': 1,
 'redness_of_eyes': 2,
```

```
 'sinus_pressure': 3,
 'runny_nose': 4,
 'congestion': 5,
 'slurred_speech': 6,
 'loss_of_smell': 7,
 'increased_appetite': 8,
 'polyuria': 9,
 'receiving_blood_transfusion': 10,
 'receiving_unsterile_injections': 11,
 'coma': 12,
 'stomach_bleeding': 13,
 'palpitations': 14}

def get_predicted_value(patient_symptoms):
  input_value=np.zeros(len(symDICT))
  for item in patient_symptoms:
    input_value[symDICT[item]]=1
  predicted_dis = mnb.predict([input_value])[0]
  print(predicted_dis)
  key = next((k for k, v in diseases_list.items() if v == predicted_dis),
None)

  return predicted_dis,key

selected_features[2]
symDICT.keys()


val=input("Enter symptoms: ")
user_symptoms=[s.strip() for s in val.split(',')]
user_symptoms=[s.strip("[]' ") for s in user_symptoms ]
predicted_disease,predictedID=get_predicted_value(user_symptoms)

print("PRED ID:")
print(type(predictedID))
desc,prcc,med,diet,work=get_info(predicted_disease)


for i in range(len(X_test)):
      actual_dis = y_test.iloc[i]
      actualID = next((k for k, v in diseases_list.items() if v ==
actual_dis), None)
      if predictedID == actualID: #Compare the predicted id with the actual
id.
        print(f"Actual: {actual_dis} | Predicted: {predicted_disease}")
        break

print("*****Predicted Disease*****")
print(predicted_disease)

print("*****Description*****")
print(desc)

print("*****Precautions*****")
print(prcc)

import ast
print("*****Medications*****")
my_med = ast.literal_eval(med[0])
i=1
```

```
for x in my_med:
  print(i, ": ",x)
  i+=1

print("*****Diets*****")
my_diet=ast.literal_eval(diet[0])
i=1
for x in my_diet:
  print(i, ": ",x)
  i+=1

print("*****Workout*****")
i=1;
for x in work:
  print(i, ": ",x)
  i+=1

import sklearn
print(sklearn.__version__)
```

File: main.py

```
from flask import Flask,request,render_template
import numpy as np
import pandas as pd
import pickle
import ast
import random


#loading datasets
sym_des = pd.read_csv("datasets/symptoms_df.csv")
precaution = pd.read_csv("datasets/precautions_df.csv")
workout = pd.read_csv("datasets/workout_df.csv")
description = pd.read_csv("datasets/description.csv")
medication = pd.read_csv("datasets/medications.csv")
diets = pd.read_csv("datasets/diets.csv")


#model
mnb=pickle.load(open("models/mnb.pkl","rb"))

diseases_list = {15: 'Fungal infection', 4: 'Allergy', 16: 'GERD', 9:
'Chronic cholestasis', 14: 'Drug Reaction', 33: 'Peptic ulcer diseae', 1:
'AIDS', 12: 'Diabetes ', 17: 'Gastroenteritis', 6: 'Bronchial Asthma', 23:
'Hypertension ', 30: 'Migraine', 7: 'Cervical spondylosis', 32: 'Paralysis
(brain hemorrhage)', 28: 'Jaundice', 29: 'Malaria', 8: 'Chicken pox', 11:
'Dengue', 37: 'Typhoid', 40: 'hepatitis A', 19: 'Hepatitis B', 20:
'Hepatitis C', 21: 'Hepatitis D', 22: 'Hepatitis E', 3: 'Alcoholic
hepatitis', 36: 'Tuberculosis', 10: 'Common Cold', 34: 'Pneumonia', 13:
'Dimorphic hemmorhoids(piles)', 18: 'Heart attack', 39: 'Varicose veins',
26: 'Hypothyroidism', 24: 'Hyperthyroidism', 25: 'Hypoglycemia', 31:
'Osteoarthristis', 5: 'Arthritis', 0: '(vertigo) Paroymsal  Positional
Vertigo', 2: 'Acne', 38: 'Urinary tract infection', 35: 'Psoriasis', 27:
'Impetigo'}
```

```
symptoms_dict = {'itching': 0, 'skin_rash': 1, 'nodal_skin_eruptions': 2,
'continuous_sneezing': 3, 'shivering': 4, 'chills': 5, 'joint_pain': 6,
'stomach_pain': 7, 'acidity': 8, 'ulcers_on_tongue': 9, 'muscle_wasting':
10, 'vomiting': 11, 'burning_micturition': 12, 'spotting_ urination': 13,
'fatigue': 14, 'weight_gain': 15, 'anxiety': 16, 'cold_hands_and_feets':
17, 'mood_swings': 18, 'weight_loss': 19, 'restlessness': 20, 'lethargy':
21, 'patches_in_throat': 22, 'irregular_sugar_level': 23, 'cough': 24,
'high_fever': 25, 'sunken_eyes': 26, 'breathlessness': 27, 'sweating': 28,
'dehydration': 29, 'indigestion': 30, 'headache': 31, 'yellowish_skin': 32,
'dark_urine': 33, 'nausea': 34, 'loss_of_appetite': 35,
'pain_behind_the_eyes': 36, 'back_pain': 37, 'constipation': 38,
'abdominal_pain': 39, 'diarrhoea': 40, 'mild_fever': 41, 'yellow_urine':
42, 'yellowing_of_eyes': 43, 'acute_liver_failure': 44, 'fluid_overload':
45, 'swelling_of_stomach': 46, 'swelled_lymph_nodes': 47, 'malaise': 48,
'blurred_and_distorted_vision': 49, 'phlegm': 50, 'throat_irritation': 51,
'redness_of_eyes': 52, 'sinus_pressure': 53, 'runny_nose': 54,
'congestion': 55, 'chest_pain': 56, 'weakness_in_limbs': 57,
'fast_heart_rate': 58, 'pain_during_bowel_movements': 59,
'pain_in_anal_region': 60, 'bloody_stool': 61, 'irritation_in_anus': 62,
'neck_pain': 63, 'dizziness': 64, 'cramps': 65, 'bruising': 66, 'obesity':
67, 'swollen_legs': 68, 'swollen_blood_vessels': 69, 'puffy_face_and_eyes':
70, 'enlarged_thyroid': 71, 'brittle_nails': 72, 'swollen_extremeties': 73,
'excessive_hunger': 74, 'extra_marital_contacts': 75,
'drying_and_tingling_lips': 76, 'slurred_speech': 77, 'knee_pain': 78,
'hip_joint_pain': 79, 'muscle_weakness': 80, 'stiff_neck': 81,
'swelling_joints': 82, 'movement_stiffness': 83, 'spinning_movements': 84,
'loss_of_balance': 85, 'unsteadiness': 86, 'weakness_of_one_body_side': 87,
'loss_of_smell': 88, 'bladder_discomfort': 89, 'foul_smell_of urine': 90,
'continuous_feel_of_urine': 91, 'passage_of_gases': 92, 'internal_itching':
93, 'toxic_look_(typhos)': 94, 'depression': 95, 'irritability': 96,
'muscle_pain': 97, 'altered_sensorium': 98, 'red_spots_over_body': 99,
'belly_pain': 100, 'abnormal_menstruation': 101, 'dischromic _patches':
102, 'watering_from_eyes': 103, 'increased_appetite': 104, 'polyuria': 105,
'family_history': 106, 'mucoid_sputum': 107, 'rusty_sputum': 108,
'lack_of_concentration': 109, 'visual_disturbances': 110,
'receiving_blood_transfusion': 111, 'receiving_unsterile_injections': 112,
'coma': 113, 'stomach_bleeding': 114, 'distention_of_abdomen': 115,
'history_of_alcohol_consumption': 116, 'fluid_overload.1': 117,
'blood_in_sputum': 118, 'prominent_veins_on_calf': 119, 'palpitations':
120, 'painful_walking': 121, 'pus_filled_pimples': 122, 'blackheads': 123,
'scurring': 124, 'skin_peeling': 125, 'silver_like_dusting': 126,
'small_dents_in_nails': 127, 'inflammatory_nails': 128, 'blister': 129,
'red_sore_around_nose': 130, 'yellow_crust_ooze': 131}
app = Flask(__name__)


def helper(dis):
  desc=description[description['Disease']==dis]['Description']
  desc=" ".join([x for x in desc])


pr=precaution[precaution['Disease']==dis][['Precaution_1','Precaution_2','P
recaution_3','Precaution_4']]
  pr=[x for x in pr.values]

  medic=medication[medication['Disease']==dis]['Medication']
  medic=[x for x in medic.values]

  diet=diets[diets['Disease']==dis]['Diet']
  diet=[x for x in diet.values]
```

```
  work=workout[workout['disease']==dis]['workout']

  return desc,pr,medic,diet,work

symDICT= {'pain_behind_the_eyes': 0,
 'throat_irritation': 1,
 'redness_of_eyes': 2,
 'sinus_pressure': 3,
 'runny_nose': 4,
 'congestion': 5,
 'slurred_speech': 6,
 'loss_of_smell': 7,
 'increased_appetite': 8,
 'polyuria': 9,
 'receiving_blood_transfusion': 10,
 'receiving_unsterile_injections': 11,
 'coma': 12,
 'stomach_bleeding': 13,
 'palpitations': 14}

def get_predicted_value(patient_symptoms):
  input_value=np.zeros(len(symDICT))
  for item in patient_symptoms:
    input_value[symDICT[item]]=1
  predicted_dis = mnb.predict([input_value])[0]
  # print(predicted_dis)
  key = next((k for k, v in diseases_list.items() if v == predicted_dis),
None)

  return predicted_dis,key


#routes
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == "POST":

        selected_symptoms = request.form.getlist('symptoms')  # Correct way
to get multiple selected options
        # invalid_symptoms = [sym for sym in selected_symptoms if sym not
in symptoms_dict]
        # if invalid_symptoms:
            # return render_template('index.html', mymsg="Please select a
symptom and try again.")

        if not selected_symptoms:
            return render_template('index.html', mymsg="Please select at
least one symptom.")

        symptoms = ','.join(selected_symptoms)  # Convert list to comma-
separated string
        print("Symptoms to pass to model:", symptoms)

        # if symptoms not in symptoms_dict:
```

```
            # return render_template('index.html',mymsg="Please select a
symptom and try again.")

        user_symptoms = [s.strip() for s in symptoms.split(',')]
        user_symptoms = [s.strip("[]' ") for s in user_symptoms]
        print(user_symptoms)
        predicted_disease , predictedID= get_predicted_value(user_symptoms)
        print(predicted_disease)
        # if(predicted_disease in diseases_list):
            # return render_template('contact.html')

        desc, pr, med, diet, work = helper(predicted_disease)

        my_prec=[]
        for i in pr[0]:
            my_prec.append(i)

        my_med= ast.literal_eval(med[0])
        my_diet= ast.literal_eval(diet[0])

        return render_template('index.html',
predicted_disease=predicted_disease,disDesc=desc,disPrec=my_prec,disMed=my_
med,disDiet=my_diet,disWork=work)


@app.route('/about')
def about():
    return render_template('about.html')
#
@app.route('/contact')
def contact():
    return render_template('contact.html')
#
@app.route('/developer')
def developer():
    return render_template('developer.html')

@app.route('/blog')
def blog():
    return render_template('blog.html')

#python main
if __name__ == '__main__':
    app.run(debug=True)
```

File: index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Home</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.c
ss" rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
      <style>
          body {
               background-image: url("{{ url_for('static',
filename='back.png') }}");
              background-size: cover;
              background-repeat:no-repeat;
            }
          .logo{
              color:black;
              margin-top:0;
              margin-bottom:3px;
              margin-right: 8px;
          }
          .mylogo{
              width: 60px;
              height: 60px;
              border-radius: 40%;
              scale: 105%;
          }
          #solvebutton{
              transition: color,background-color 0.25s ease-out;
          }
          #solvebutton:hover{
              color:red;
              background-color: white;
              transition: background-color,color 0.25s ease-in;
          }
          #disID{
                background-color:#F39334;
                color:black;
              transition: background-color,color 0.25s ease-out;
            }
          #descID{
              background:#268AF3 ;
              color:black
              transition: background-color,color 0.25s ease-out
          }
            #disID:hover,#descID:hover{
                color:red;
              background-color: white;
              transition: background-color,color 0.25s ease-in;
            }
          #loading {
    font-size: 18px;
    color: #555;
    text-align: center;
```

```
    margin: 20px;
  }

  #results {
    transition: opacity 3s ease-in-out;
      transition-timing-function: step-end ;
  }
  select {
      width: 100%;
      height: 180px;
      padding: 10px;
      font-size: 16px;
      border: 2px solid #ccc;
      border-radius: 8px;
      background-color: #f9f9f9;
      box-shadow: inset 0 1px 3px rgba(0, 0, 0, 0.1);
      transition: border-color 0.3s, box-shadow 0.3s;
    }
  option{
      color:#dc3545;
  }
  option:hover{
      color:white;
      background-color: #dc3545;
      transition: color,background-color 0.2s ease-in;
  }
  option:checked {
      background-color: #dc3545;
      color: white;
    }
  .form-container {
            /*max-width: 500px;*/
            margin: auto;
            background: #ffffff;
            padding: 20px;
            border-radius: 10px;
             box-shadow: 8px 10px 2px grey;
       }
      </style>
      <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/choices.js/public/assets/styles/choices.
min.css" />
  </head>
  <body>
    <!--    navbar-->
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
      <div class="logo">
         <a href="/"> <img class="mylogo"
src="{{url_for('static',filename='logo.jpg')}}"></a>
      </div>
    <a class="navbar-brand" href="/">Symptom Solver</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarSupportedContent" aria-
controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarSupportedContent">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
```

```
        <li class="nav-item">
          <a class="nav-link active" aria-current="page" href="/">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="/about">About</a>
        </li>

          <li class="nav-item">
          <a class="nav-link" href="/contact">Contact</a>
        </li>

      </ul>

    </div>
  </div>
</nav>
    {% if not predicted_disease and not mymsg%}
    <h1 class="text-center mt-4">Welcome!</h1>

    <div class="container mt-4 form-container" name="divform"
style="background: rgb(33, 37, 41); color: red;border-radius: 13px;padding:
25px">
        <form action="/predict" method="post">
            <div class="form-group">
                <label for="symptoms" style="padding: 2px 2px 20px 2px;
color: white">Select symptoms</label>
                     <select name="symptoms" id="symptoms" multiple>
                    <option value="pain_behind_the_eyes">Pain behind the
eyes</option>
                    <option value="throat_irritation">Throat
irritation</option>
                    <option value="redness_of_eyes">Redness of
eyes</option>
                    <option value="sinus_pressure">Sinus pressure</option>
                    <option value="runny_nose">Runny nose</option>
                    <option value="congestion">Congestion</option>
                    <option value="slurred_speech">Slurred speech</option>
                    <option value="loss_of_smell">Loss of smell</option>
                    <option value="increased_appetite">Increased
appetite</option>
                    <option value="polyuria">Polyuria</option>
                    <option value="receiving_blood_transfusion">Receiving
blood transfusion</option>
                    <option
value="receiving_unsterile_injections">Receiving unsterile
injections</option>
                    <option value="coma">Coma</option>
                    <option value="stomach_bleeding">Stomach
bleeding</option>
                    <option value="palpitations">Palpitations</option>
            </select>
            </div>
            <br>
            <div class="text-center">
            <button class="btn btn-danger" id="solvebutton" style="border-
radius: 30px; width:100px; padding: 14px; margin-bottom:7px ; font-weight:
bold;">Solve</button>
            </div>
        </form>
    </div>
```

```
    {% endif %}

<div id="loading" style="display:none;">
  <h2 class="text-center mt-4">Analyzing symptoms... please wait.</h2>
</div>

<div id="invalid-msg" style="display:none; color: red; text-align: center;
font-weight: bold; font-size: 30px; margin-top: 15px">
    {{mymsg}}
    <div>
         <button style="border-radius: 30px padding:4px; margin: 5px 40px
5px 0; font-size:20px;font-weight:bold; width:140px;
background:white;color:black;"><a href="/">Go Back</a></button>
    </div>
</div>

<div class="CONTAIN" id="results" style="display:none;">


    {% if predicted_disease %}

<!-- Results -->
<h1 class="text-center my-4 mt-4">Our System Results</h1>
<div class="container">

    <div class="result-container text-center">
        <!-- Buttons to toggle display -->
        <div>

        <button id="disID" class="toggle-button" data-bs-toggle="modal"
data-bs-target="#diseaseModal" style="padding:4px;  margin: 5px 40px 5px 0;
font-size:20px;font-weight:bold; width:140px; border-radius:5px;
">Disease</button>
            </div>
        <div>
        <button id="descID" class="toggle-button mywait" data-bs-
toggle="modal" data-bs-target="#descriptionModal" style="padding:4px;
margin: 5px 40px 5px 0; font-size:20px;font-weight:bold; width:140px;
border-radius:5px; ;">Description</button>
         </div>
        <button class="toggle-button mywait" data-bs-toggle="modal" data-
bs-target="#precautionModal" style="padding:4px; margin: 5px 40px 5px 0;
font-size:20px;font-weight:bold; width:140px; border-radius:5px;
background:#F371F9 ;color:black;">Precaution</button>
        <button class="toggle-button mywait" data-bs-toggle="modal" data-
bs-target="#medicationsModal" style="padding:4px; margin: 5px 40px 5px 0;
font-size:20px;font-weight:bold; width:140px;border-radius:5px;
background:#F8576F ;color:black;">Medications</button>
        <button class="toggle-button mywait" data-bs-toggle="modal" data-
bs-target="#workoutsModal" style="padding:4px; margin: 5px 40px 5px 0;
font-size:20px;font-weight:bold; width:140px; border-radius:5px;
background:#99F741 ;color:black;">Workouts</button>
        <button class="toggle-button mywait" data-bs-toggle="modal" data-
bs-target="#dietsModal" style="padding:4px; margin: 5px 40px 5px 0; font-
size:20px;font-weight:bold; width:140px; border-radius:5px;
background:#E5E23D;color:black;">Diets</button>
        <div>
            <div></div>
<!--            <button><a href="index.html">Go Back</a></button>-->
            <div></div>
```

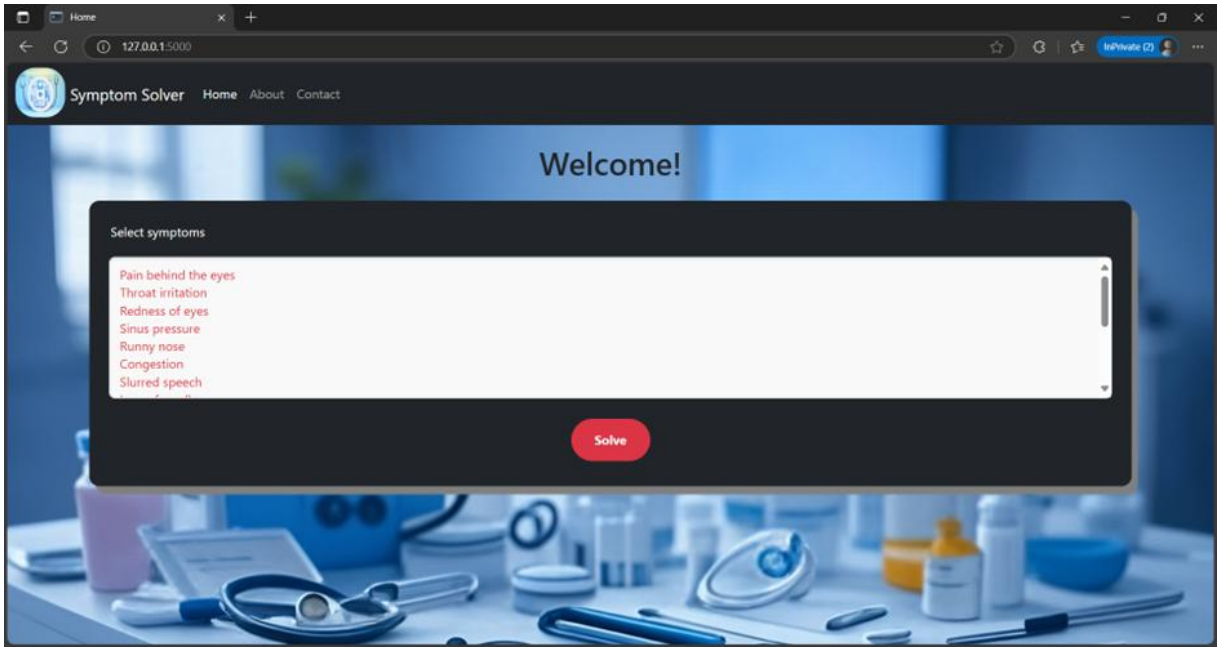
```
        </div><br>
        <div>
            <div></div>
            <button style="border-radius: 30px padding:4px; margin: 5px
40px 5px 0; font-size:20px;font-weight:bold; width:140px;
background:white;color:black;"><a href="/">Go Back</a></button>
            <div></div>
        </div>
    </div>

</div>

{% endif %}

<!-- Disease Modal -->
    <div class="modal fade" id="diseaseModal" tabindex="-1" aria-
labelledby="diseaseModalLabel" aria-hidden="true">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header" style="background-color: #020606;
color:white;"> <!-- Set header background color inline -->
                    <h5 class="modal-title"
id="diseaseModalLabel">Predicted Disease</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body" style="background-color: #modal-
body-color;"> <!-- Set modal body background color inline -->
                    <p>{{ predicted_disease }}</p>
                </div>
            </div>
        </div>
    </div>


    <!-- Description Modal -->
    <div class="modal fade" id="descriptionModal" tabindex="-1" aria-
labelledby="descriptionModalLabel" aria-hidden="true">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header" style="background-color: #020606;
color:white;">
                    <h5 class="modal-title"
id="descriptionModalLabel">Description</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body">
                    <p>{{ disDesc }}</p>
                </div>
            </div>
        </div>
    </div>

<!-- Precaution Modal -->
    <div class="modal fade" id="precautionModal" tabindex="-1" aria-
labelledby="precautionModalLabel" aria-hidden="true">
        <div class="modal-dialog">
            <div class="modal-content">
```

```
                <div class="modal-header" style="background-color: #020606;
color:white;">
                    <h5 class="modal-title"
id="precautionModalLabel">Precaution</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body">
                    <ul>
                        {% for i in disPrec %}
                            <li>{{ i }}</li>
                        {% endfor %}
                    </ul>
                </div>
            </div>
        </div>
    </div>

    <!-- Medications Modal -->
    <div class="modal fade" id="medicationsModal" tabindex="-1" aria-
labelledby="medicationsModalLabel" aria-hidden="true">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header" style="background-color: #020606;
color:white;">
                    <h5 class="modal-title"
id="medicationsModalLabel">Medications</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body">
                    <ul>
                        {% for i in disMed %}
                            <li>{{ i }}</li>
                        {% endfor %}
                    </ul>
                </div>
            </div>
        </div>
    </div>
    <!-- Workouts Modal -->
    <div class="modal fade" id="workoutsModal" tabindex="-1" aria-
labelledby="workoutsModalLabel" aria-hidden="true">
        <div class="modal-dialog" >
            <div class="modal-content">
                <div class="modal-header" style="background-color: #020606;
color:white;">
                    <h5 class="modal-title"
id="workoutsModalLabel">Workouts</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body">
                    <ul>
                        {% for i in disWork %}
                            <li>{{ i }}</li>
                        {% endfor %}
                    </ul>
                </div>
            </div>
```

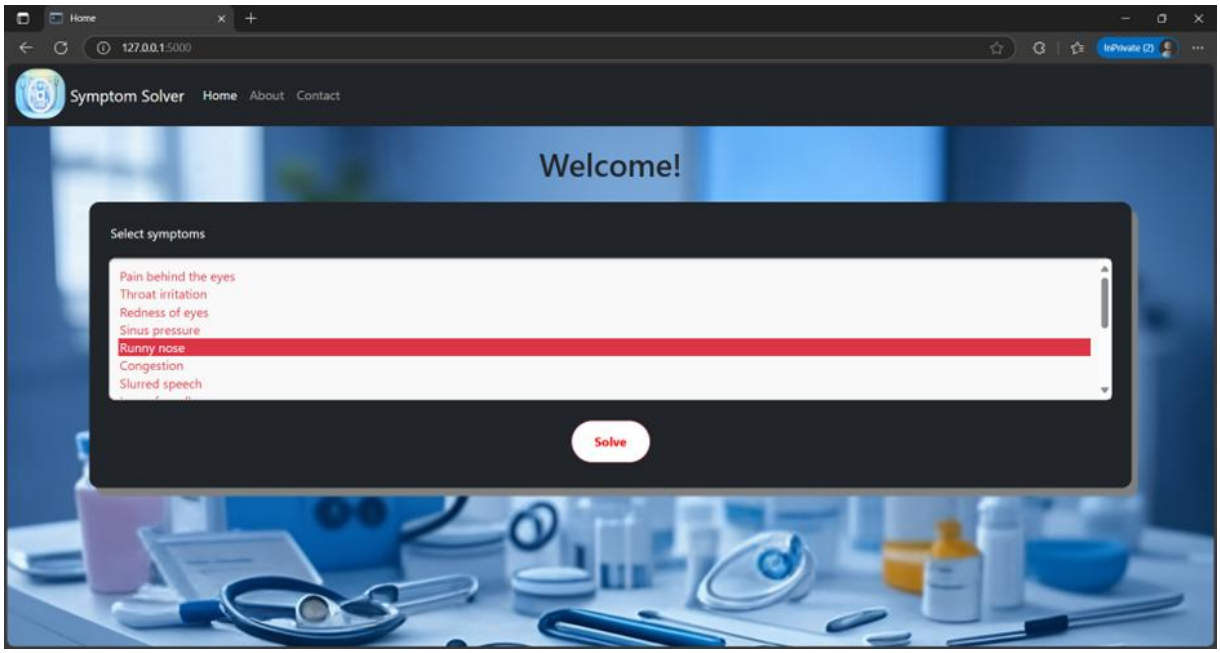
```
        </div>
    </div>

    <!-- Diets Modal -->
    <div class="modal fade" id="dietsModal" tabindex="-1" aria-
labelledby="dietsModalLabel" aria-hidden="true">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header" style="background-color: #020606;
color:white;">
                    <h5 class="modal-title" id="dietsModalLabel">Diets</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body">
                    <ul>
                        {% for i in disDiet %}
                            <li>{{ i }}</li>
                        {% endfor %}
                    </ul>
                </div>
            </div>
        </div>
    </div>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.
min.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
    <script
src="https://cdn.jsdelivr.net/npm/choices.js/public/assets/scripts/choices.
min.js"></script>
    <script>
window.onload = function () {
    const resultsDiv = document.getElementById("results");
    const loadingDiv = document.getElementById("loading");
    const invalidMsgDiv = document.getElementById("invalid-msg");
    {% if mymsg %}
      loadingDiv.style.display = "block";
      setTimeout(function () {
        loadingDiv.style.display = "none";
        invalidMsgDiv.style.display = "block";
      }, 2000); // Delay for invalid symptom
    {% elif predicted_disease %}
      loadingDiv.style.display = "block";

      setTimeout(function () {
        loadingDiv.style.display = "none";
        resultsDiv.style.display = "block";
      }, 2000); // Delay for valid result
    {% endif %}
  };
    </script>
  <style>

  </style>
  </body>
</html>
```

Screenshots

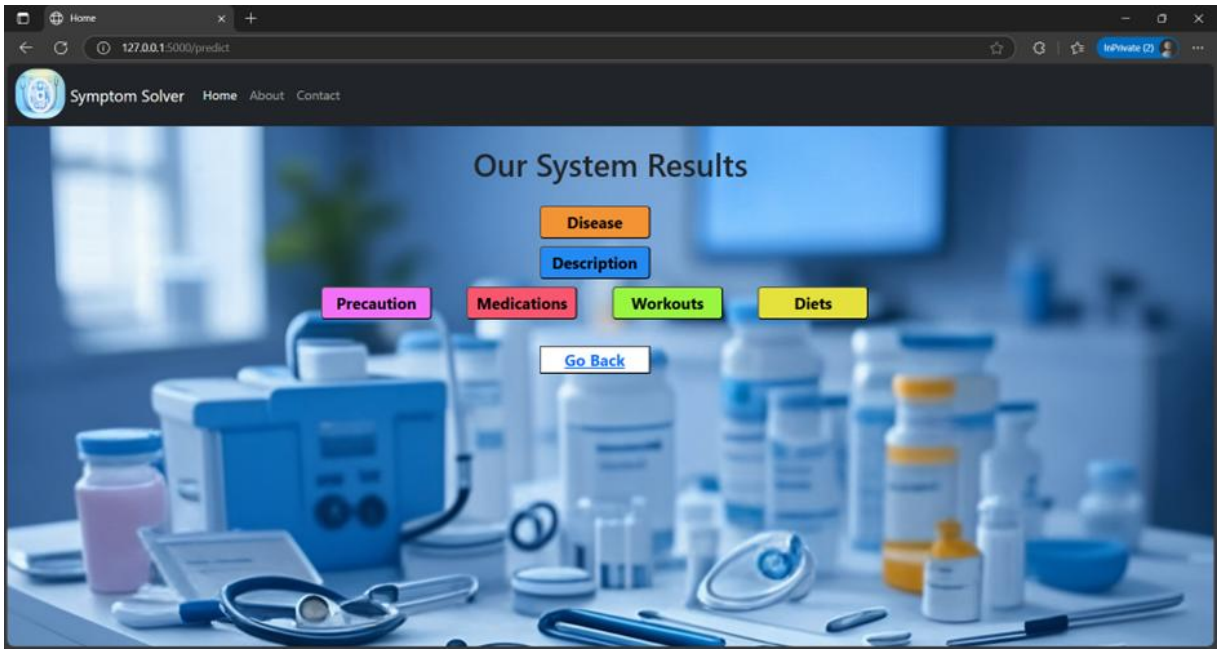
Homepage:



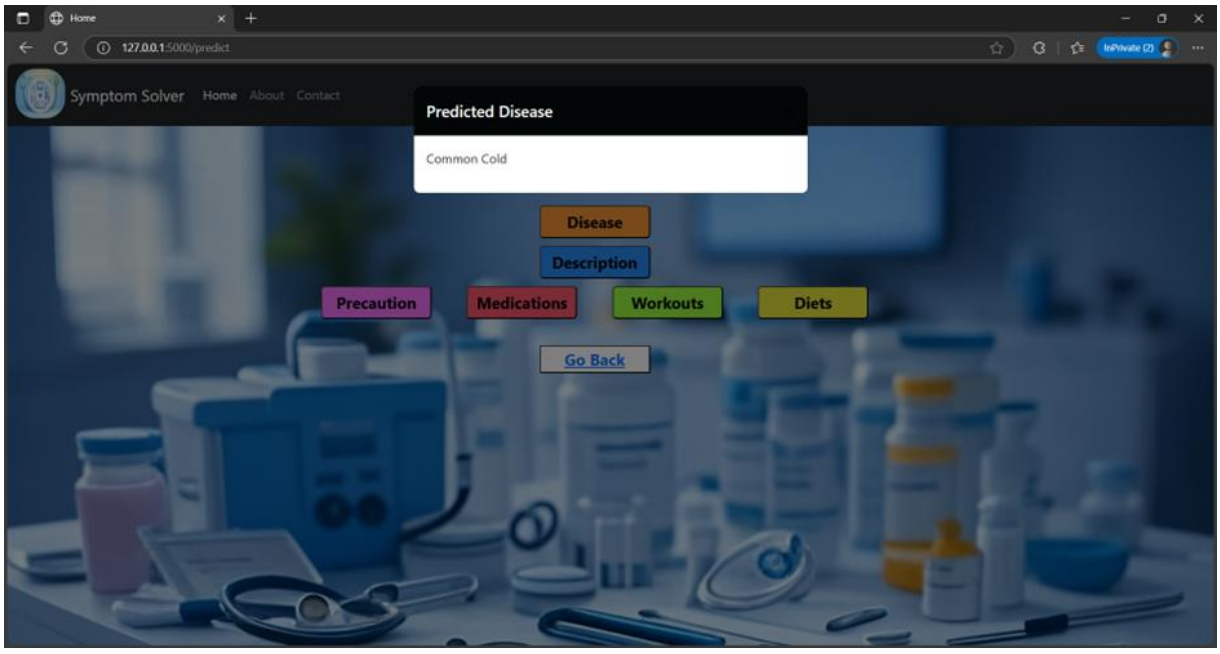
Choosing symptoms:



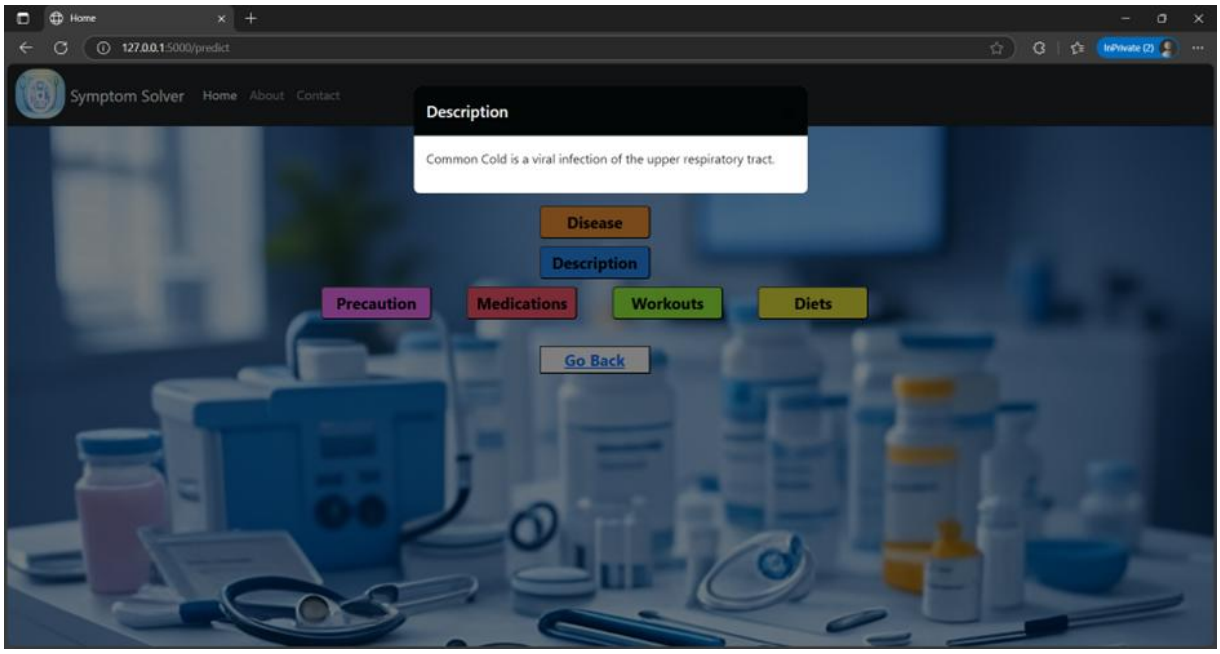
System Results:



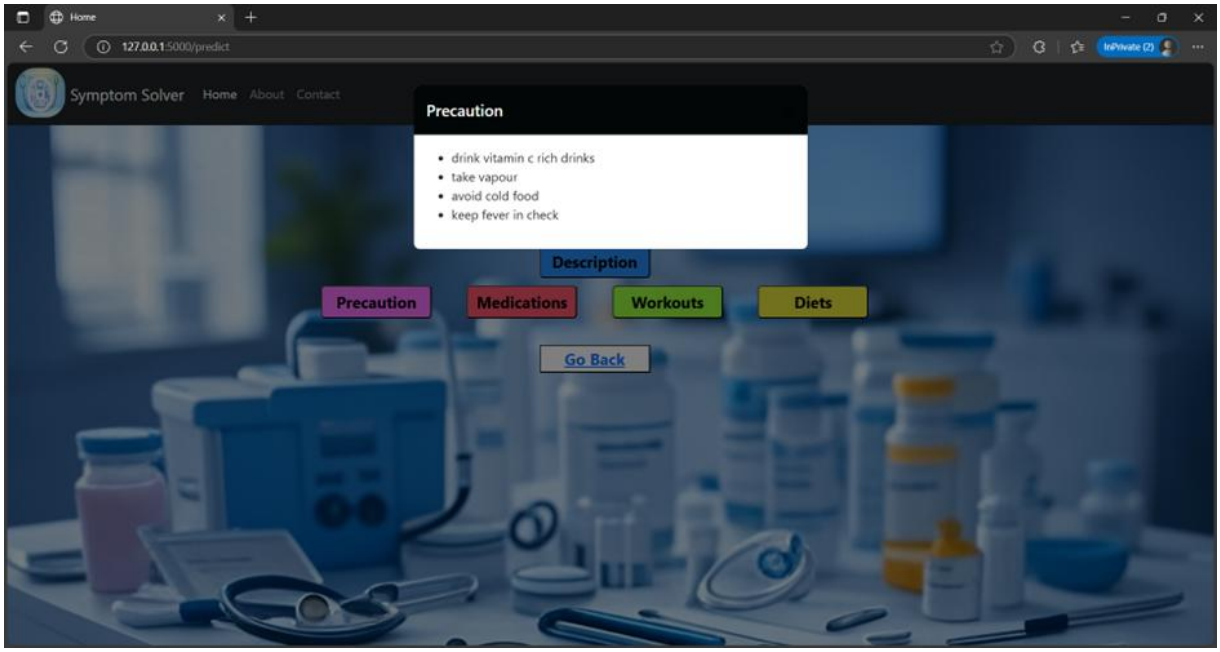
Predicted Disease:



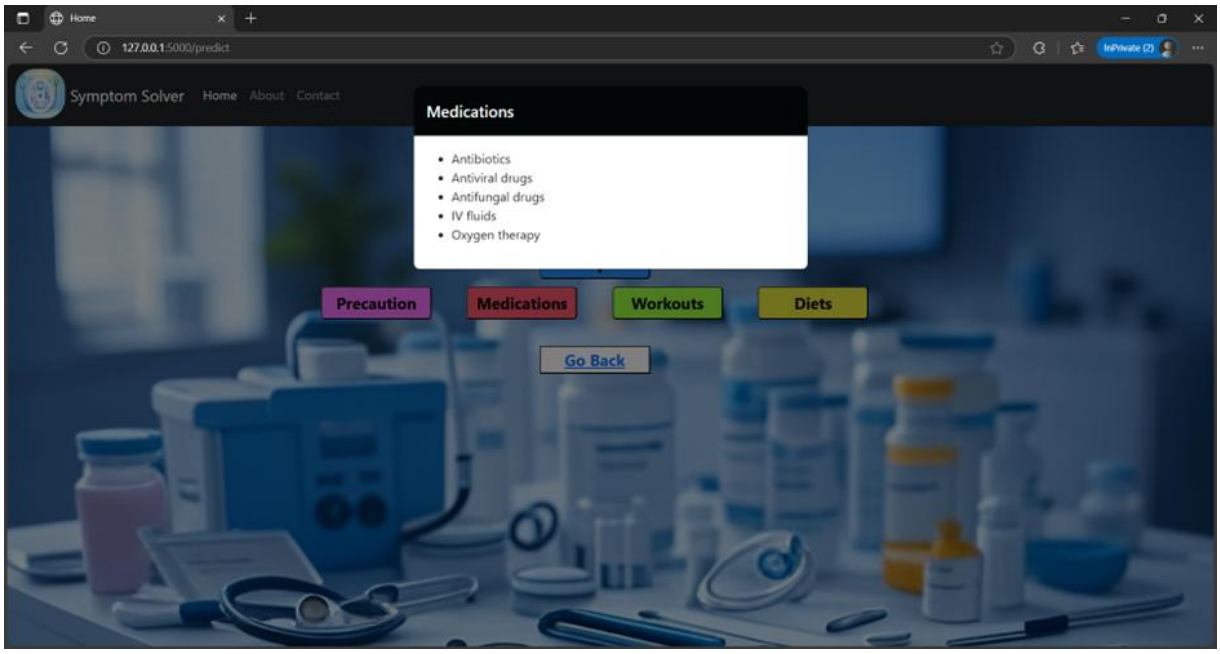
Description:



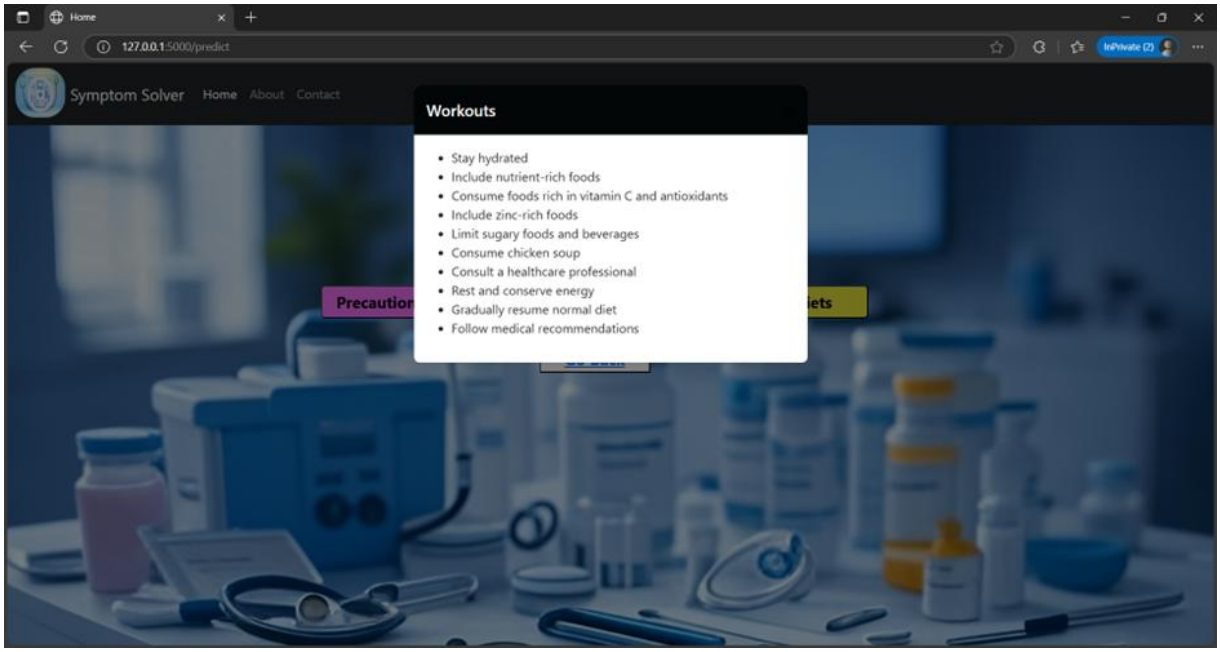
Precautions:



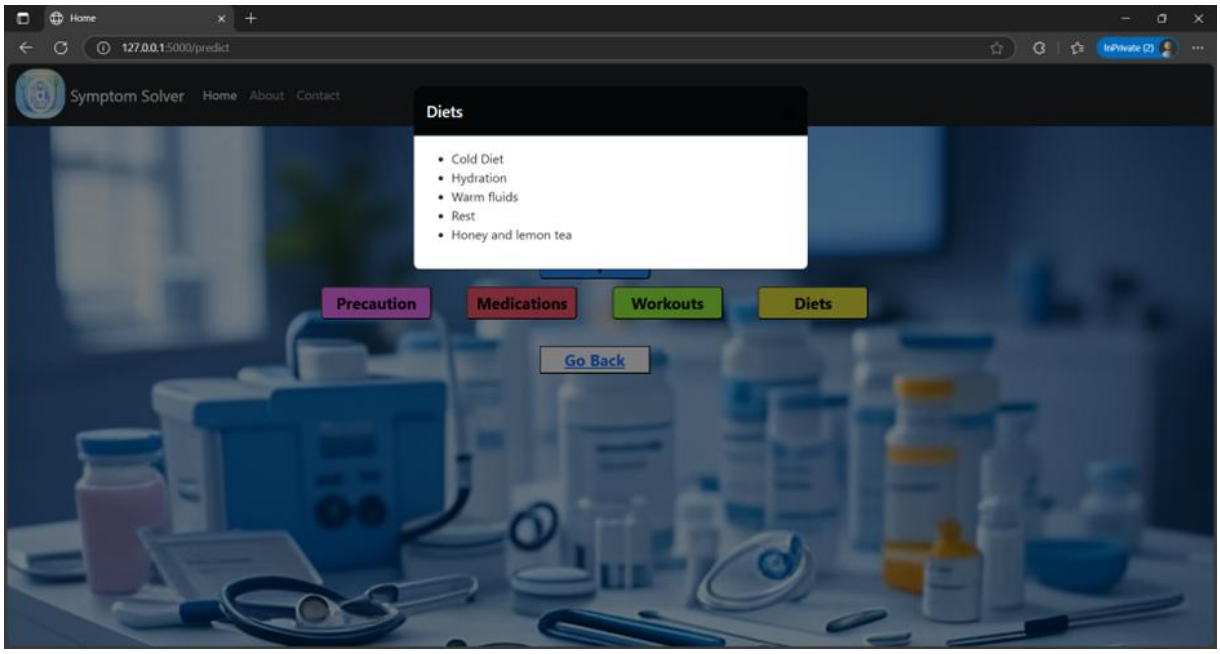
Medications:



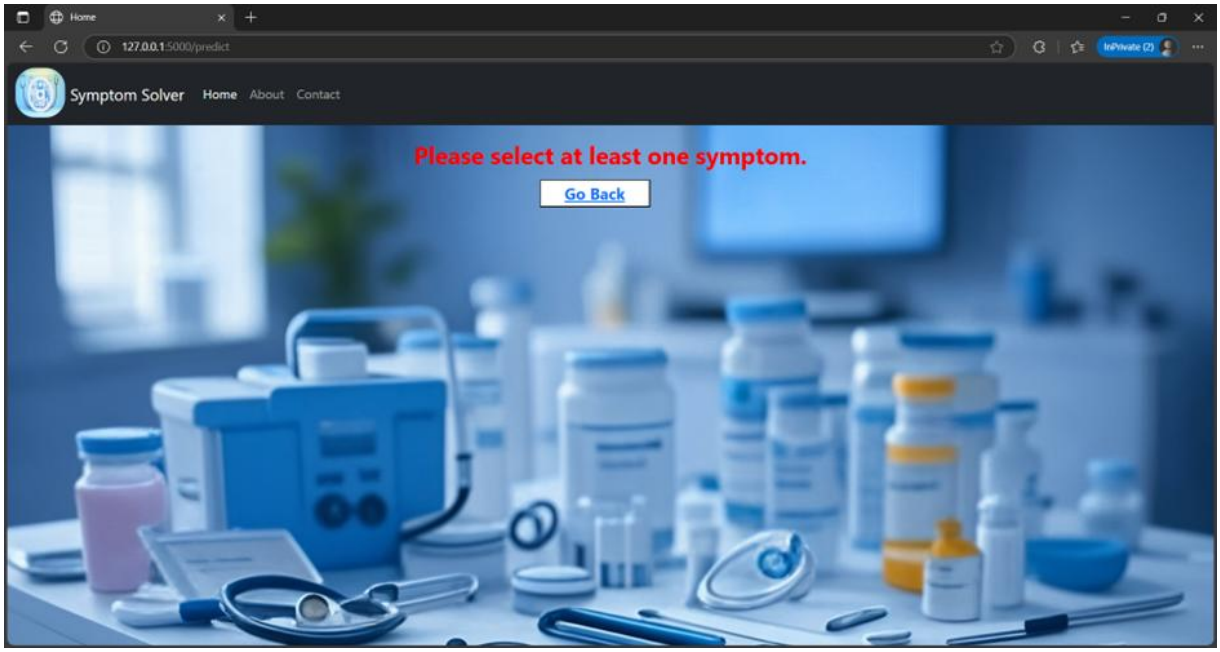
Workouts:



Diets:



No symptom is selected:



Conclusion and Future Scope

Conclusion:

The Symptom Solver project represents a significant step toward utilizing data science and machine learning in the healthcare domain. The application allows users to input a list of symptoms and, based on a trained predictive model, returns the most likely disease or health condition. By leveraging Python's robust libraries such as Pandas, NumPy, and Scikit-learn, along with a dataset of symptoms and diseases, this system delivers quick and intelligent predictions, improving accessibility to preliminary medical insights.

Through this project, we have successfully demonstrated how machine learning algorithms can be trained on structured datasets to extract meaningful patterns and perform classification tasks. The prediction model achieved considerable accuracy during testing and has shown consistent performance with a range of user inputs. The interface allows for intuitive interaction, and the modular design makes it both scalable and maintainable.

Additionally, the project has underscored the importance of preprocessing in machine learning, such as handling missing values, normalizing data, and evaluating multiple models for better results. Various classification algorithms were experimented with—such as Support Vector Machine, and Naive Bayes—and their performance was evaluated to select the most optimal model.

Overall, the Symptom Solver system simplifies the early diagnosis process and promotes awareness about health conditions based on visible symptoms. While not a replacement for professional medical consultation, it can serve as a helpful preliminary diagnostic tool.

Future Scope:

Despite its current capabilities, the Symptom Solver project opens up numerous opportunities for enhancement and broader applicability. Below are several directions in which the project can be extended:

1. **Integration with Real-Time Databases**

   Future versions can fetch real-time medical data, such as outbreak information or seasonal disease trends, from online health databases (e.g., WHO, CDC) to enhance prediction accuracy and relevance.

2. **Incorporation of NLP for Free-Text Input**

   Currently, the system expects structured input. A natural progression would be the use of Natural Language Processing (NLP) to allow users to describe their symptoms in free text, making the tool more user-friendly and versatile.

3. **Severity and Duration-Based Predictions**

   Including symptom severity (mild/moderate/severe) and duration would allow for more granular diagnosis and help differentiate between diseases with similar symptom profiles.

4. **User Profile and Medical History Tracking**

   Adding user profiles and historical symptom tracking can lead to more personalized predictions and even early detection of chronic conditions.

5. **Multilingual Support**

   To increase accessibility, especially in rural or non-English speaking areas, the system can be developed with multilingual support.

6. **Mobile App Development**

   Deploying the Symptom Solver as a mobile application would significantly improve its reach and utility. Users could access the tool anytime, anywhere, making it a true point-of-care assistant..

7. **Model Improvement via Deep Learning**

   With a richer dataset, deep learning models like neural networks can be implemented to improve accuracy and handle more complex symptom combinations.

8. **Validation with Medical Experts**

   Collaborating with medical professionals to validate the predictions can improve trust and help in fine-tuning the model for real-world deployment.

By continuously evolving with user feedback, technological advancements, and medical research, the Symptom Solver has the potential to become a vital component in the digital healthcare ecosystem. The fusion of artificial intelligence with healthcare not only empowers individuals but also paves the way for smarter, data-driven decision-making in medicine.

References

- Keniya, R., Khakharia, A., Shah, V., Gada, V., Manjalkar, R., Thaker, T., ... & Mehendale, N. (2020). Disease prediction from various symptoms using machine learning. Available at SSRN 3661426.
- Leong, J. Y., & Booma, P. M. (2020). Symptom-based disease prediction system using machine learning. J. Theor. Appl. Inf. Technol, 98, 19.
- Uddin, M. Z., Dysthe, K. K., Følstad, A., & Brandtzaeg, P. B. (2022). Deep learning for prediction of depressive symptoms in a large textual dataset. Neural Computing and Applications, 34(1), 721-744.